
Inventory and Supply *Management* System

*Cross-platform inventory and supply management ERP built with
full-stack microservices*

ABSTRACT. This document describes the design and implementation of a full-stack ERP system for inventory and supply management. The system is built as two sibling monorepos: a backend of nine independent microservices (Java/Spring, C#/.NET, and Node/TypeScript) fronted by two API gateways, communicating over REST and Apache Kafka; and a frontend monorepo containing an Expo/React Native mobile application (the primary client) plus a Module-Federation web shell with seven micro-frontends. The platform manages companies, fleets, warehouses, products, inventory, and the full order lifecycle, with a domain rule that stock is only ever created through order closure. Cross-cutting concerns include a shared Slovenian/English internationalization package built on a live recursive proxy, HMAC-signed JWT authentication with gateway header stamping, a resilient Kafka-driven notification service, a barcode/QR scanner with a scan-to-create-product flow, and an optional Azure OpenAI analysis service with a deterministic template fallback. This text targets a software engineer or thesis reviewer and explains every project-specific mechanism in full.

AUTHOR

MARKO SPASOVSKI

DATE

28.05.2026

Contents

Table of contents

01	Executive summary
02	System overview and architecture
03	Backend services
04	API gateways
05	Inter-service communication
06	Mobile application (Expo / React Native)
07	Internationalization (i18n)
08	Authentication and authorization
09	Order lifecycle
10	Stock (inventory) flows
11	Scanner subsystem
12	Notifications subsystem
13	AI analysis
14	Offline and resilience
15	Date pickers
16	Build / run
17	Testing
18	Known constraints and design tradeoffs

19 Appendix A — QR payload schema

20 Appendix B — Mermaid diagrams (consolidated)

21 Appendix C — Glossary

Section 01

Executive summary

The Inventory and Supply Management System is a full-stack enterprise resource planning platform whose central purpose is to track the movement of goods between companies, warehouses, and inventory positions while enforcing a small set of strict domain invariants. The most important of these invariants is that inventory stock is never created or incremented by direct user action: stock only ever increases as a side effect of an order reaching the `Closed` state. This rule shapes the entire user interface and several backend interactions, and it is referenced repeatedly throughout this document.

The platform is organized into two version-control repositories. The backend repository contains nine microservices, each owning its own PostgreSQL database (or no database, in the case of the stateless AI service), plus two gateways. Three implementation stacks are used deliberately: Java with Spring Boot for the auth, company, fleet, warehouse, and inventory services; C# with .NET 8 for the product and order services; and Node.js with TypeScript for the notification and AI services and for the mobile gateway. Services coordinate through synchronous REST calls for queries and through Apache Kafka topics for domain events.

The frontend repository is an npm-workspaces monorepo. The primary client is an Expo/React Native mobile application targeting field operators; a secondary web client is a Module-Federation host shell composed of seven micro-frontends. Both clients share three framework-agnostic packages: a domain package (roles, query keys, the order-status enum), a types-only DTO package, and an internationalization package providing parallel Slovenian and English message tables.

The remainder of this document treats each subsystem in turn. Particular attention is given to mechanisms that were non-obvious to implement correctly: the live i18n proxy and the locale-driven remount that makes it propagate into already-mounted navigation screens; the once-only JWT refresh interceptor that coalesces concurrent unauthorized responses; the synchronous scanner guard that prevents a fast camera callback from firing the same scan many times; and the notification service's ordered, failure-tolerant RxJS pipeline.

Section 02

System overview and architecture

2.1 Monorepo layout

The system lives in two sibling git repositories.

The **frontend monorepo** (`Inventory-Supply-Management-Frontend/`) is an npm-workspaces project. Its `mobile/` workspace is the Expo/React Native application and is the primary client. Its `web/` workspace is a Module-Federation host shell composed with seven micro-frontends: `auth-mf`, `warehouses-mf`, `inventory-mf`, `orders-mf`, `companies-mf`, `fleet-mf`, and `products-mf`. Three shared packages sit under `packages/`: `domain` holds framework-agnostic runtime values and types (roles, React Query keys, the order-status enum); `api-types` holds types-only DTOs with no runtime code; and `i18n` holds the pure Slovenian/English message tables together with `getStrings(locale)` and the `Locale` type. The repository root carries an `.npmrc` with `legacy-peer-deps=true` to unblock a React 19 peer-dependency mismatch on `react-test-renderer`.

The **backend monorepo** (`Inventory-Supply-Management-Backend/`) places its nine microservices under `services/`, the mobile gateway under `mobile-gateway/`, and the web gateway under `gateway-service/`. A `compose.yaml` file describes the docker-compose topology for databases, the Kafka broker, the gateway, and the micro-frontends; most application services run on host networking during development.

2.2 Runtime topology

The mobile application never talks to a service directly. It addresses a single base URL — the mobile gateway — and the gateway routes each request to the appropriate downstream service after validating the caller's token. The web shell talks to the separate web gateway, which has a different shape suited to the micro-frontend composition. Domain events flow asynchronously: the order and inventory services publish to Kafka, and the notification service is the sole consumer, fanning resolved notifications out to clients over a WebSocket.

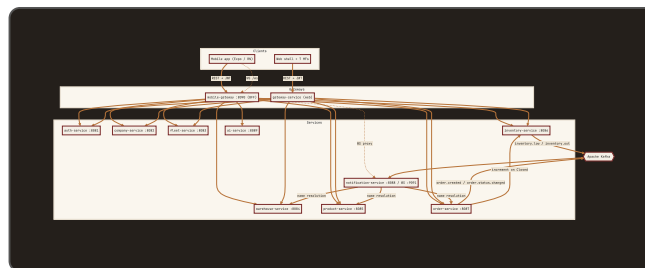


FIGURE 2.1 – RUNTIME TOPOLOGY: CLIENTS, GATEWAYS, SERVICES, AND EVENT FLOW

2.3 Network boundaries

The trust boundary is the gateway. Clients hold a JWT and present it on every request; the gateway is the component that verifies the signature. Downstream services generally do not re-verify the token — instead they trust the `X-User-Id`, `X-User-Email`, and `X-User-Role` headers that the gateway stamps after a successful verification. The notification service is an exception in one respect: it independently validates the JWT on WebSocket upgrade, because the upgrade request arrives with the token as a query parameter rather than as a stamped header. Internal REST calls between services (for example, the notification service resolving names against the product, warehouse, and order services) cross only the internal network and are not individually authenticated in this design.

Section 03

Backend services

Each service is independently deployable, owns its persistence, and exposes a focused REST surface. The table below is the canonical service matrix.

SERVICE	PORT	STACK	DATABASE (PORT)	RESPONSIBILITY
auth-service	8081	Java / Spring Boot	Postgres (5432) users	Register, login, refresh; issues HMAC-signed JWTs; stamps the role claim.
company-service	8082	Java / Spring	Postgres (5433) companies	CRUD companies, contact info, country/city.
fleet-service	8083	Java / Spring	Postgres (5434) fleet	CRUD drivers and vehicles, vehicle types.
warehouse-service	8084	Java / Spring	Postgres (5435) warehouses	CRUD warehouses with capacity (total/used).
product-service	8085	C# / .NET 8	Postgres (5436) products	CRUD products and categories; GET /products/by-sku .
inventory-service	8086	Java / Spring	Postgres (5437) inventory	Inventory rows with min/max thresholds; consume (multipart with PDF). Emits inventory.low / inventory.out .
order-service	8087	C# / .NET 8	Postgres (5438) orders	Order CRUD; status transitions; multipart delivery-document upload on close. Emits order events. On Closed , increments inventory.
notification-service	8088 · WS 9091	Node/TS + RxJS + KafkaJS + WS	Postgres (5439) notifications	Consumes Kafka events, resolves IDs to names (cached 5 min), persists notifications, broadcasts over WebSocket. Resilient bootstrap.
ai-service	8089	Node/TS	(no DB)	Aggregates inventory/product/warehouse

SERVICE	PORT	STACK	DATABASE (PORT)	RESPONSIBILITY
				snapshots; totals, alerts, reorder suggestions; optional Azure OpenAI summary with template fallback.
mobile-gateway	8090	Node/Express + http-proxy-middleware	(stateless)	BFF for mobile: validates JWT, stamps X-User-* headers, proxies REST and WebSocket, applies CORS and rate limiting.

3.1 auth-service

The authentication service is the root of trust. It is a Spring Boot application backed by a PostgreSQL `users` database. It exposes registration, login, and refresh endpoints. On successful authentication it issues a JSON Web Token signed with HMAC-SHA-256 using a base64-encoded secret key. The token carries the user's identity and, critically, a `role` claim. Because the role claim is embedded at signing time, every other component in the system can make authorization decisions without consulting the auth service again, provided it trusts the signature (the gateways) or trusts the gateway's stamped headers (downstream services).

3.2 company-service

A straightforward Spring CRUD service over a `companies` database. It manages company records along with their contact information and country/city fields. Companies are referenced by orders (an order is placed for a company) and surface in notification messages by name.

3.3 fleet-service

A Spring CRUD service over a `fleet` database managing drivers and vehicles, including vehicle types. It supports the `DRIVER` role's domain and the logistics side of supply management.

3.4 warehouse-service

A Spring service over a `warehouses` database. Each warehouse carries a total capacity and a used capacity. These two figures drive the capacity-warning interaction during order approval (described in §9) and are part of the snapshot the AI service aggregates.

3.5 product-service

A .NET 8 service over a `products` database, responsible for products and their categories. It exposes a SKU lookup at `GET /products/by-sku`, which the scanner relies on to determine whether a scanned SKU already exists. One behavior of this service is load-bearing for the mobile client: `POST /categories` historically returned an empty `Ok()` body (or a zero GUID), so the mobile client cannot always read the new category's identifier from the create response and must fall back to refetching and resolving by name (see §11).

3.6 inventory-service

A Spring service over an `inventory` database. Inventory rows carry minimum and maximum thresholds. The minimum threshold drives low-stock badging in the UI and the `inventory.low` Kafka event. The service exposes a `consume` (issue) operation implemented as a multipart request that requires an attached PDF document representing the consumption record. The service emits `inventory.low` and `inventory.out` events to Kafka as stock crosses thresholds or reaches zero. Inventory rows are created and incremented only by the order service when an order closes; there is no public “add stock” endpoint exercised by the UI.

3.7 order-service

A .NET 8 service over an `orders` database. It owns order records and their status transitions. The transition into `Closed` is special: it accepts a multipart upload of a delivery-document PDF at `POST /orders/upload-document`, then sets the order to `Closed`, and as a side effect instructs the inventory service to increment the relevant inventory. It publishes `order.created` when an order is first created and `order.status.changed` on every subsequent transition.

3.8 notification-service

A Node/TypeScript service combining RxJS, KafkaJS, a `ws` WebSocket server, and a PostgreSQL `notifications` database. It is the only Kafka consumer in the system. Its responsibilities are to translate raw domain events into human-readable notifications by resolving identifiers to names (calling the product, warehouse, and order services, with a five-minute cache), to persist those notifications, and to broadcast them to connected clients. Its bootstrap is deliberately resilient: the HTTP read API starts listening before the Kafka connection is attempted, and a failure to connect to Kafka logs an error rather than crashing the process. This is covered in detail in §12.

3.9 ai-service

A stateless Node/TypeScript service with no database. It aggregates current snapshots from the inventory, product, and warehouse services and computes totals, low-stock alerts, and reorder suggestions. If the `AZURE_OPENAI_ENDPOINT`, `AZURE_OPENAI_API_KEY`, and `AZURE_OPENAI_DEPLOYMENT` environment variables are present, it asks Azure OpenAI for a natural-language summary and labels the result `source: "azure"`; otherwise it emits a deterministic template summary labelled `source: "template"`. It is gated to managers and above at the gateway (see §13).

Section 04

API gateways

The system has two gateways because the two clients have different composition needs. The **mobile gateway** is a backend-for-frontend (BFF) tailored to the mobile app, and the **web gateway** (`gateway-service`) serves the Module-Federation web shell.

4.1 mobile-gateway

The mobile gateway is a Node/Express application using `http-proxy-middleware`. It runs on port 8090 and is the single address the mobile app knows. Its responsibilities are: **JWT validation** — every incoming REST request must carry `Authorization: Bearer <jwt>`, whose HMAC-SHA-256 signature it verifies; **header injection** — after verifying the token it stamps `X-User-Id`, `X-User-Email`, and `X-User-Role` onto the proxied request; **REST proxying** by path to the correct downstream; **WebSocket proxying** of `/ws` upgrades to the notification service on port 9091 (configured with `ws: true`); **CORS**, open in development to ease physical-device testing across tunneled origins; and **rate limiting**, capped at 300 requests per 15-minute window.

```
// mobile-gateway: after verifying the JWT
const claims = jwt.verify(token, SECRET); // HMAC-SHA-256
req.headers["x-user-id"] = claims.sub;
req.headers["x-user-email"] = claims.email;
req.headers["x-user-role"] = claims.role; // gates downstream
next();
```

TS

4.2 gateway-service (web)

The web gateway serves the host shell and its micro-frontends. It shares the JWT verification responsibility with the mobile gateway but has a different routing shape because the web shell composes micro-frontends rather than addressing a single BFF surface. The web shell stores the JWT and shares the active locale across micro-frontends via `localStorage` plus a `'storage'` event, because Module Federation cannot share React runtime singletons across separately built remotes.

4.3 Manager gating

The AI analysis route is additionally restricted at the gateway by a `requireManager` middleware that admits only the `MANAGER` role and above. The `X-User-*` headers are forwarded to the AI service so it has the caller’s identity, but the authorization decision is made at the boundary.

Section 05

Inter-service communication

Two communication styles coexist. Queries that need an immediate answer use synchronous REST. Domain facts that other parts of the system care about — an order was created, stock fell low — are published as Kafka events and consumed asynchronously.

5.1 Kafka topics

TOPIC	PRODUCER	PAYLOAD KEY FIELDS
<code>order.created</code>	order-service	<code>orderId</code> , <code>companyId</code> , <code>companyName</code> , <code>warehouseId</code> , <code>status</code> , <code>createdAt</code>
<code>order.status.changed</code>	order-service	<code>orderId</code> , <code>oldStatus</code> , <code>newStatus</code> , <code>changedAt</code>
<code>inventory.low</code>	inventory-service	<code>warehouseId</code> , <code>capacityLeft</code> , <code>productId?</code>
<code>inventory.out</code>	inventory-service	<code>warehouseId</code> , <code>productId?</code>

The sole consumer of all four topics is the notification service.

5.2 The notification event pipeline

When the notification service receives an event, it must turn opaque identifiers into text a human can read. An `order.status.changed` event, for instance, contains an `orderId` but no order description; a useful notification needs the company or order name. The service therefore performs an asynchronous transform that calls the product, warehouse, and order services to resolve those identifiers, then persists and broadcasts the resulting message. The full pipeline mechanics — ordered persistence, caching, and failure degradation — are described in §12.

5.3 The name-resolver pattern

The name resolver is the component responsible for turning identifiers into human-readable names. It caches successful lookups for five minutes so that a burst of events about the same warehouse or product does not produce a burst of REST calls. It deliberately never caches failures, so a transient outage of a downstream service is not pinned into the cache for five minutes; the next event simply retries. Each lookup is bounded by a three-second `AbortController` timeout, and if resolution fails the message degrades to short-identifier wording rather than being dropped. The principle is that a notification is never lost merely because a name could not be resolved.

Section 06

Mobile application (Expo / React Native)

The mobile application is the primary client and the most feature-complete surface of the system.

6.1 Platform and stack

The app runs on Expo SDK 54 with React Native 0.80 and uses `expo-router` for file-system-based routing. It is written entirely in TypeScript. State is managed with `zustand` plus `zustand/persist` over `AsyncStorage`, split into three stores: one for authentication, one for locale, and one for the toast queue. Server state is handled by `@tanstack/react-query` v5 with a `staleTime` of 60 seconds, a `gcTime` of 5 minutes, and `retry: 1`. HTTP uses `axios` with interceptors (see §8). A hand-written `ReconnectingWsClient` manages the notification WebSocket with exponential backoff from one second to a sixty-second cap and a three-failure threshold after which the UI falls back to 30-second polling. Forms use `react-hook-form` with `zod`, where validation messages are stored as i18n keys (for example `auth.emailInvalid`) so that module-level zod schemas do not freeze to whichever locale was active when the module loaded.

Hardware and platform integrations include `expo-camera`'s `CameraView` (with `barcodeTypes: ['ean13', 'ean8', 'code128', 'qr']`) for scanning, `expo-document-picker` for the close-order and consume PDF uploads, best-effort `expo-notifications` token registration (which works only in dev builds, not in Expo Go on SDK 53+), `@react-native-community/datetimestpicker` wrapped in a single reusable `DatePickerField`, and `@react-native-community/netinfo` driving an `OfflineBanner`.

6.2 File-system routing layout

The app uses `expo-router`'s file-system routing. The route tree below is the authoritative map; route groups in parentheses do not appear in the URL.

```
app/
  _layout.tsx           # ErrorBoundary > SafeAreaProvider > QueryClient > AuthGuard > <Stack> (keyed on locale to force language)
  (auth)/
    login.tsx
    register.tsx
  (tabs)/
    _layout.tsx         # Tabs: home / stock / orders / notifications / more
    index.tsx           # Dashboard / Home
    stock/
      index.tsx         # Stock list, low-stock badges, edit-thresholds
      consume.tsx       # Multi-item consume with required PDF document
    orders/
      index.tsx         # Filterable order list
      [id].tsx          # Order detail + status transitions (capacity w/ stock)
      create.tsx        # New order form (today → +1y delivery date picker)
    notifications.tsx   # WS-driven list, mark-read, polling-fallback if disconnected
    more/
      index.tsx         # Menu (Products, Warehouses, Fleet, Companies, Settings)
      profile.tsx
      scanner.tsx       # Barcode/QR scanner + scan-to-create-product flow
      ai.tsx            # AI analysis acknowledge + summary view
      products/{index,[id],new,categories}.tsx
      warehouses/{index,[id]}.tsx
      fleet/{index,drivers/[id],vehicles/[id]}.tsx
      companies/{index,[id]}.tsx
```

6.3 Tab structure and per-screen responsibilities

The authenticated experience is a five-tab navigator. **Home** is a dashboard. **Stock** (*Zaloga*) lists inventory with low-stock badges, lets the operator edit min/max thresholds, and provides the consume flow.

Orders (*Naročilo*) lists and filters orders, opens order detail with its status transitions, and creates new orders. **Notifications** (*Obvestilo*) shows the WebSocket-driven list with mark-read and a polling-fallback indicator. **More** is a menu surface that branches into Products, Warehouses (*Skladišče*), Fleet, Companies, the Scanner (*Skener*), the AI analysis screen, and the profile.

6.4 State management

The three zustand stores are persisted to AsyncStorage so that authentication and locale survive app restarts. React Query is the cache for server state; its 60-second `staleTime` and 5-minute `gcTime` double as a modest offline read cache, allowing recently fetched lists to render while the device is briefly offline (see §14).

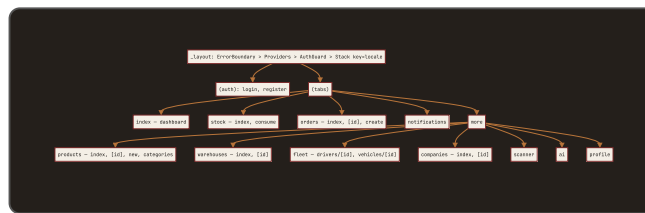


FIGURE 6.1 — MOBILE ROUTE MAP (EXPO-ROUTER FILE-SYSTEM ROUTING)

Section 07

Internationalization (i18n)

The system ships with parallel Slovenian and English message tables and supports switching locale at runtime. The implementation is more involved than a typical i18n setup because React Navigation memoizes mounted screens, which defeats a naive locale toggle.

7.1 The shared @erp/i18n package

The shared package defines `Locale = 'sl' | 'en'` and a canonical `Strings = typeof sl` type, where `sl.ts` is the source of truth and `en.ts` must mirror its shape exactly so that the two tables are structurally identical. It exports `getStrings(locale)` to retrieve a table, along with `LOCALES`, `LOCALE_LABEL`, and `DEFAULT_LOCALE`. The package is pure data and functions with no framework dependency, so both the mobile app and the web shell consume it.

7.2 The live recursive proxy `sl`

The mobile-side facade lives in `constants/i18n.ts`. It exports a symbol named `sl` of type `Strings`, but `sl` is not the Slovenian table directly. It is a recursive `Proxy` that, on every property access, resolves the property against the strings table for the currently active locale, read live from the locale store:

```
// constants/i18n.ts - recursive live proxy
const make = (path: string[]): any =>
  new Proxy(() => {}, {
    get(_t, key: string) {
      const table = getStrings(useLocaleStore.getState().locale);
      const node = path.reduce((o, k) => o?.[k], table)?.[key];
      return typeof node === "object" ? make([...path, key]) : node;
    },
  });
export const sl: Strings = make([]);
```

Because the active locale is read at the moment of access, an expression like `sl.orders.title` returns the title in whatever locale is active right now. Call sites do not need to change when the locale flips — they simply read `sl.something` as if it were a static table.

7.3 The `useStrings()` hook

For components that want subscription semantics — to re-render when the locale changes rather than only resolving lazily — `lib/i18n/locale.ts` exposes a `useStrings()` hook backed by the persisted zustand locale store.

7.4 The module-level pitfall and the refactor

The proxy is powerful but has a sharp edge: any value computed at **module-load time** that reads through the proxy is frozen to the locale that happened to be active when the module was first evaluated. Several values in the codebase fell into this trap — zod validation schemas, the `MoreScreen` menu array, and the login zod schema. The fix has two accepted forms. Either build the value **inside** the component body so that each render re-resolves through the proxy, or store `i18n` keys in the module-level data and resolve them at render time with a tiny `msg(key)` helper. Both patterns appear in the codebase; the menu array, for example, is now constructed inside the render so its labels always reflect the current locale.

7.5 Key-remount on locale toggle

Lazy resolution alone is not enough, because React Navigation keeps already-mounted tab screens in memory and a plain re-render of the root does not propagate into them. The root layout therefore subscribes to the locale and renders its inner navigator with `key={locale}`:

```
// app/_layout.tsx - force a full remount on locale change
const locale = useLocaleStore((s) => s.locale);
return (
  <AuthGuard>
    <Stack key={locale} /> {/* locale flip → new key → stack remounts */}
  </AuthGuard>
);
```

When the locale toggles, the key changes, expo-router's stack remounts, and every screen re-resolves the proxy from scratch. This was the only reliable way to push a locale change into screens that React Navigation had already memoized.

Section 08

Authentication and authorization

8.1 Login, register, and the JWT

Authentication begins at the auth service, which on a successful login or registration issues a JSON Web Token signed with HMAC-SHA-256 using a base64-encoded shared secret. The token carries the user's identity and a `role` claim. Both the mobile gateway and the notification service hold the same secret, so each can verify the signature independently. The role claim is the authority for all gating decisions: the gateway re-stamps it as `X-User-Role` for downstream services that do not verify the token themselves.

8.2 The refresh-once axios interceptor

The mobile app's axios instance has a request interceptor that injects `Authorization: Bearer <jwt>` and a response interceptor that handles expiry. On a `401`, the interceptor calls `POST /auth/refresh` exactly once with the stored refresh token, then retries the original request. Crucially, concurrent `401`s

share a single in-flight refresh promise so that a screen issuing several requests at once does not trigger several parallel refreshes:

```
TS
let refreshing: Promise<string> | null = null;
api.interceptors.response.use(undefined, async (err) => {
  if (err.response?.status !== 401 || err.config._retried) throw err;
  err.config._retried = true;
  refreshing ??= refresh(getRefreshToken()).finally(() => (refreshing = null));
  const token = await refreshing; // concurrent 401s await the same promise
  err.config.headers.Authorization = `Bearer ${token}`;
  return api(err.config); // retry once
});
```

If the refresh itself fails, the stored credentials are cleared and the user is returned to the login flow.

8.3 Role-based UI gating

Four roles exist: `ADMIN`, `MANAGER`, `WORKER`, and `DRIVER`. They are mapped to UI features in `mobile/lib/auth/features.ts`. Components query capability with the `useHasFeature(feature)` hook or wrap protected UI in a `<RoleGate feature="X">` component. UI gating is a usability measure; the authoritative enforcement happens at the gateway and within services that read `X-User-Role`.

8.4 Gateway verification

The gateway is where the JWT is actually verified. It checks the HMAC signature, and on success it stamps `X-User-Id`, `X-User-Email`, and `X-User-Role` onto the downstream request. The manager-only AI route is enforced here through a `requireManager` middleware.

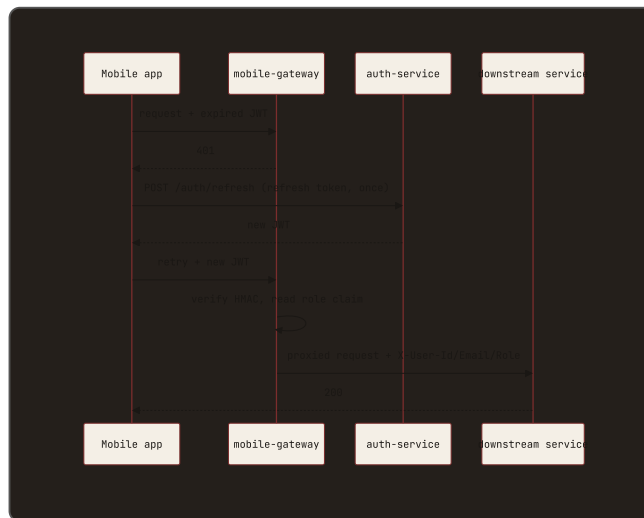


FIGURE 8.1 – JWT VERIFICATION AND THE REFRESH-ONCE SEQUENCE

Section 09

Order lifecycle

An order (*Naročilo*) moves through a small state machine. From **Requested** it is either **Approved** or **Rejected**. From **Approved** it becomes **Delivered**. From **Delivered** it becomes **Closed**, but only after a delivery-document PDF has been uploaded. Closing an order is the single mechanism by which inventory stock increases.

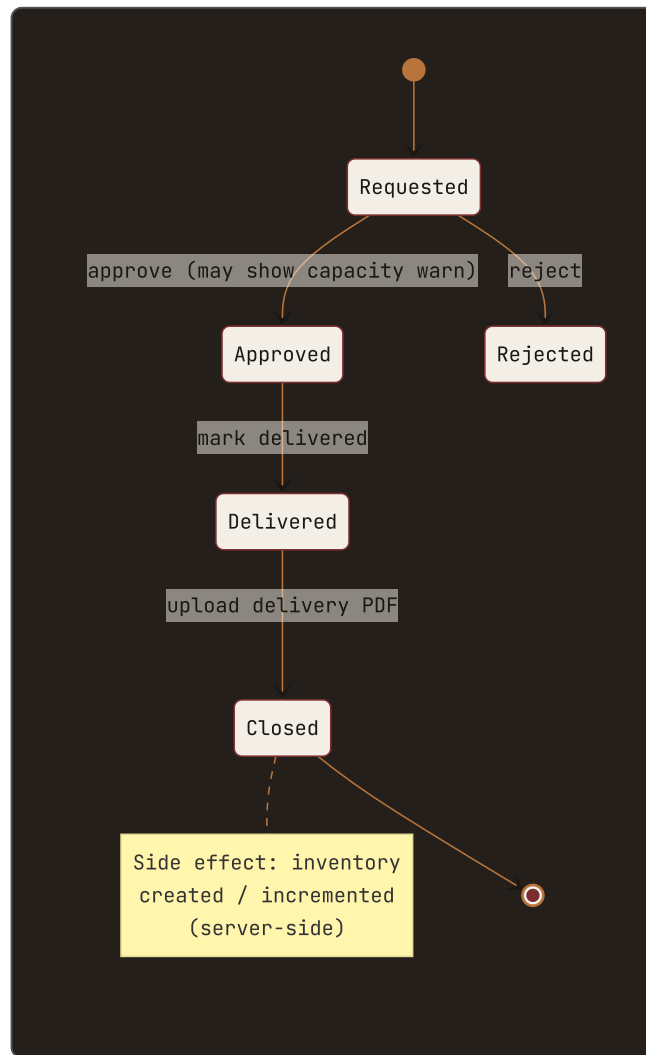


FIGURE 9.1 — ORDER STATE MACHINE

9.1 The capacity-warn flow on approve

When an operator approves an order, the UI compares the order quantity against the destination warehouse’s remaining capacity, computed as `warehouse.totalCapacity - warehouse.usedCapacity`. If the order would exceed the remaining capacity, the UI presents a three-button alert — Cancel, Reject, or Approve anyway — before any request is sent. This lets the operator make an informed decision rather than silently overcommitting a warehouse.

9.2 Delivered to Closed and the mandatory document

The transition from `Delivered` to `Closed` is intercepted. The client first requires the operator to select a delivery-document PDF, which it uploads as a multipart request to `POST /orders/upload-document`. Only after the upload succeeds does the order move to `Closed`. On reaching `Closed`, the order service instructs the inventory service to create or increment the relevant inventory row. This server-side side effect is the only path by which stock grows, which is why the mobile UI offers no manual add-stock action (see §10 and §18).

Section 10

Stock (inventory) flows

The stock screen (*Zaloga*) is where operators observe and draw down inventory. It deliberately offers no way to add stock.

10.1 What the screen offers

The stock screen lets the operator filter inventory by warehouse, view the list with low-stock badging, edit each row's minimum and maximum thresholds, and consume (issue) stock. Low-stock badging is computed against the minimum threshold; when a row's quantity is at or below its minimum, the UI badges it. Editing the thresholds is consequential: the minimum threshold is what the inventory service uses to decide when to emit the `inventory.low` Kafka event, so changing it changes when low-stock notifications fire.

10.2 The consume flow

Consuming stock is a multipart operation that requires the operator to attach a PDF document representing the consumption record. The mobile consume screen supports selecting multiple items and submitting them with the required document. The reason the document is user-picked rather than generated is implementation-specific: the web `inventory-mf` auto-generates the consumption-record PDF with `jspdf`, but React Native has no Blob or jspdf pipeline. Rather than build a parallel generation path or change the backend, the mobile flow asks the operator to attach an appropriate PDF themselves — a signed delivery sheet, a scan of a paper form, and so on.

Domain rule. Stock only ever increases when an order moves to **Closed**, at which point the order service tells the inventory service to increment. Exposing a manual add-stock action would create a second, unaudited path for inventory to change and would break the invariant that every stock increase is traceable to a closed order with an attached delivery document. The absence of an add-stock button is therefore a feature, not an omission.

Section 11

Scanner subsystem

The scanner (*Skener*) reads barcodes and QR codes and, for product QR payloads, drives an in-screen flow that can create a product — and, if needed, its category — without ever navigating to a form.

11.1 The QR payload format

A product QR encodes a small JSON object:

```
{
  "t": "product",
  "name": "Kladivo",
  "sku": "KLA-001",
  "description": "Jekleno kladivo 500g",
  "weight": 0.7,
  "category": "Orodje"
}
```

JSON

A payload with `t: "product"` (or, more loosely, a `name` + `sku` pair) is treated as a create-product payload. Anything else is treated as a bare SKU and looked up.

11.2 The useBarcode ref-guard

`expo-camera`'s `CameraView.onBarcodeScanned` can fire many times per second while a code is in frame. An earlier implementation guarded against duplicate handling with React state, but state updates are asynchronous, so several callbacks slipped through before the state flipped — and each one pushed the same screen, so a single physical scan navigated several times. The fix is a synchronous

`useRef` guard that flips immediately and is checked at the top of the handler, so the scan callback runs exactly once until explicitly reset:

```
// useBarcode - synchronous guard against the multi-fire bug
const locked = useRef(false);
const onBarcodeScanned = useCallback((e) => {
  if (locked.current) return; // synchronous, no async state gap
  locked.current = true;
  handleScan(e.data);
}, [handleScan]);
const reset = () => { locked.current = false; };
```

11.3 The in-screen result state machine

The scanner screen drives a result card through an explicit state machine. On a product payload, the handler first calls `getBySku` to confirm the SKU is free, then `getAllCategories` to resolve the payload's category name to an identifier.

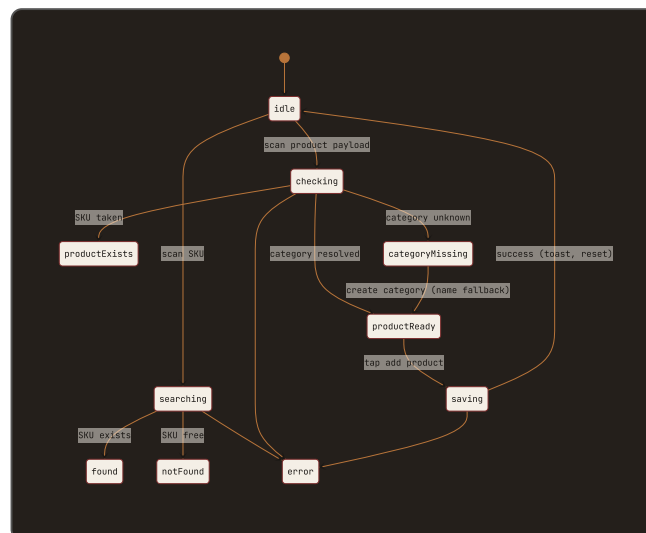


FIGURE 11.1 – SCANNER RESULT STATE MACHINE

11.4 The category create-and-resolve fallback

If the category does not exist, the result card shows a “Ustvari kategorijo” (create category) button. Tapping it sends `POST /categories` with `{ name, description: name }`, auto-filling the

description from the name. Because the product service may return an empty body or a zero GUID from the create call, the client cannot rely on reading the new identifier from the response. It therefore refetches `/categories` and resolves the category by name as a fallback, then advances the state machine to `productReady`:

```
TS
// scanner - create category, then resolve by name if the id is missing
const created = await createCategory({ name, description: name });
let id = created?.id;
if (!id || id === EMPTY_GUID) {
  const all = await refetchCategories(); // empty Ok() fallback
  id = all.find((c) => c.name === name)?.id; // resolve by name
}
setCategoryId(id);
setState("productReady");
```

Tapping “Dodaj produkt” (add product) calls `POST /products` directly — there is no navigation to a form. When the QR omits a description, it defaults to the product name. On success the screen shows a toast and resets for the next scan.

Section 12

Notifications subsystem

The notification service (*Obvestilo*) turns Kafka domain events into human-readable, persisted, broadcast notifications, and it is engineered to stay up even when its dependencies are not.

12.1 Building a notification from an event

`process-kafka-event.usecase.ts` builds one `Notification` per event. The build step is **asynchronous** because it resolves identifiers to names via REST against the product, warehouse, and order services. If a resolution fails, the message degrades to short-identifier wording so that a notification is never lost merely because a name lookup failed.

12.2 The RxJS pipeline and why `concatMap`

The pipeline is `event$.pipe(concatMap(build), concatMap(save), tap(broadcast))`. The persistence step uses `concatMap`, not `switchMap`. This choice is deliberate: because the upstream is

asynchronous, `switchMap` would cancel an in-flight database write the moment the next event arrived, which would drop notifications under load. `concatMap` processes events one at a time in order, guaranteeing each persistence completes before the next begins.

```
// notification-service - ordered, cancellation-free pipeline
event$
  .pipe(
    concatMap(build),      // async name resolution, in order
    concatMap(save),      // never cancels an in-flight DB write
    tap(broadcast),       // push to connected WebSocket clients
  )
  .subscribe();
```

```
graph LR
    A[kafka events] --> B[concatMap: build (async, resolve names)]
    B --> C[concatMap: save (ordered, no cancellation)]
    C --> D[tap: broadcast over WebSocket]
    D --> E[Connected clients]
```

FIGURE 12.1 — NOTIFICATION RXJS EVENT PIPELINE

12.3 The name resolver

`name-resolver.ts` caches successful lookups for five minutes and never caches failures, so a transient outage is not pinned. Each REST call is bounded by a three-second `AbortController` timeout.

12.4 Resilient bootstrap

The bootstrap order is: `initDb` → `build deps` → `wire pipeline` → `app.listen(8088)` → `try connect Kafka` `catch log`. The HTTP server listens **before** the Kafka connection is attempted, specifically so that a Kafka outage cannot take down the read API at `GET /notifications`. A Kafka connection failure is logged, not thrown — the process keeps running and serves persisted notifications even with no live event stream.

12.5 WebSocket delivery

The service runs a `ws` server on port 9091 and validates the JWT on upgrade, reading the token from a `?token=...` query parameter (an upgrade request cannot carry an `Authorization` header through all clients reliably). The mobile gateway proxies `/ws` to this server using `http-proxy-middleware` with

`ws: true`. On the client, the `ReconnectingWsClient` reconnects with exponential backoff and, after three consecutive failures, signals the UI to fall back to 30-second polling so notifications keep flowing even when the broker or socket is unavailable.

Section 13

AI analysis

The AI analysis screen offers managers a snapshot-based assessment of inventory health with an optional natural-language summary.

13.1 Gateway gating

The route is gated to `MANAGER` and above by the gateway's `requireManager` middleware. The caller's `X-User-*` headers are forwarded to the AI service, but the authorization decision is made at the boundary.

13.2 What the service computes

The AI service fetches current inventory, product, and warehouse snapshots and computes totals, low-stock alerts, and reorder suggestions from them. This deterministic computation always runs regardless of whether the language model is configured.

13.3 Azure OpenAI integration and the template fallback

If `AZURE_OPENAI_ENDPOINT`, `AZURE_OPENAI_API_KEY`, and `AZURE_OPENAI_DEPLOYMENT` are all present in the environment, the service asks Azure OpenAI to produce a natural-language summary of the computed figures and returns it with `source: "azure"`. If the environment is not configured, the service produces a deterministic template summary instead and returns `source: "template"`. The client can show the same UI either way; the `source` field tells it which path produced the text.

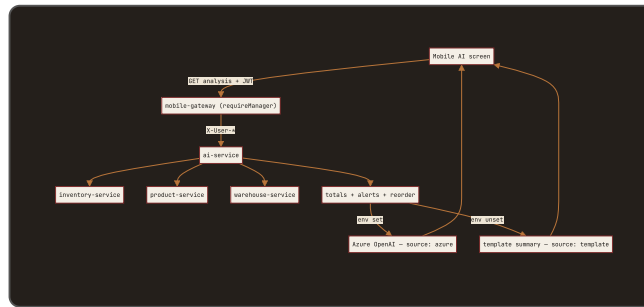


FIGURE 13.1 – AI ANALYSIS REQUEST FLOW WITH TEMPLATE FALLBACK

13.4 Mobile UI states

The screen has three states: an initial acknowledge state where the user opts into running the analysis, a loading state while the request is in flight, and a summary state showing the totals, alerts, suggestions, and the natural-language text.

Section 14

Offline & resilience

The mobile app is built for field use, where connectivity is intermittent, so it degrades gracefully rather than failing outright.

Network status is observed through `useOfflineState`, which wraps `@react-native-community/netinfo`. When the device goes offline, a top-of-screen `OfflineBanner` appears so the operator knows that what they are seeing may be cached. React Query's `staleTime` of 60 seconds and `gcTime` of 5 minutes act as the offline read cache: recently fetched lists and details remain available and render immediately while the network is down, rather than showing spinners or errors.

Authentication resilience is handled by the refresh-once interceptor described in §8: a 401 triggers a single shared refresh, a successful refresh retries the original request, and a failed refresh clears the stored credentials and falls back to the login flow. The notification WebSocket has its own resilience path — exponential backoff reconnection and a three-failure fallback to 30-second polling — so notifications continue to arrive even when the socket cannot be maintained.

Section 15

Date pickers

Date entry is consolidated into a single reusable `DatePickerField` component at `mobile/components/ui/DatePickerField.tsx`, which papers over substantial platform differences in `@react-native-community/datetimepicker`. On Android it opens the picker imperatively via `DateTimePickerAndroid.open`, presenting the native dialog. On iOS it renders the spinner inside a modal sheet. To prevent the well-known problem where dark-mode devices render the picker's digits in a near-invisible color, the component sets `themeVariant="light"` and an explicit `textColor`, so the digits stay readable regardless of the device theme. Dates are stored in the canonical `YYYY-MM-DD` form throughout the app, which is also the format the create-order screen uses for its today-to-plus-one-year delivery-date range.

Section 16

Build / run

The backend is brought up with docker-compose, with one service run by hand, and the mobile app is served by the Expo dev server, typically over a tunnel so a physical device can reach a laptop-hosted gateway.

```
BASH
# Backend
cd Inventory-Supply-Management-Backend
docker compose up -d                # DBs, Kafka, gateway, MFs
cd services/notification-service && npm run dev    # not containerised; run man

# Mobile
cd Inventory-Supply-Management-Frontend/mobile
cp .env.example .env                # set EXPO_PUBLIC_API_URL
npx expo start --tunnel --port 19000 --clear
# scan the QR with Expo Go (iOS) or the Camera app

# Cloudflared tunnel for physical-device testing against a laptop gateway
cloudflared tunnel --url http://localhost:8090
# put the https URL into mobile/.env as EXPO_PUBLIC_API_URL
```

The mobile app reads its API base URL from the `EXPO_PUBLIC_API_URL` environment variable, which must point at the mobile gateway. For physical-device testing the gateway is exposed through a public HTTPS tunnel (cloudflared, or equivalently ngrok), and that tunnel URL is written into `mobile/.env`. The notification service is intentionally not containerized in the default compose file and is run manually with `npm run dev`.

Section 17

Testing

Automated tests use Jest, on both the mobile app and the notification service. Because the frontend is an npm-workspaces monorepo importing the shared `@erp/*` packages, the Jest configuration includes a `moduleNameMapper` that resolves `@erp/*` imports to their workspace sources so tests run against the real shared code rather than stale build artifacts. The mobile test setup mocks AsyncStorage in `jest.setup.js`, since the persisted zustand stores would otherwise touch native storage that does not exist under Jest. Coverage focuses on the logic most prone to regression: the i18n resolution behavior, the scanner state transitions and ref-guard, the refresh-once interceptor, and the notification service's event-processing pipeline.

Section 18

Known constraints and design tradeoffs

Several decisions in this system were chosen consciously over plausible alternatives, and recording the reasoning is part of the documentation's purpose.

THE SL PROXY PLUS LOCALE REMOUNT, RATHER THAN THREADING STRINGS THROUGH 45 FILES

A conventional approach would pass a strings object or a `t()` function into every component and every module. With the proxy, call sites simply read `sl.section.key` and always get the active locale, and the `key={locale}` remount forces even React-Navigation-memoized screens to re-resolve. The tradeoff is the module-level freezing pitfall (§7.4), which is mitigated by building locale-dependent values inside the render or storing i18n keys and resolving them at render time. The alternative — editing dozens of files to thread a translation function everywhere — was judged more error-prone and more invasive.

NO ADD-STOCK UI

The domain rule that stock only increases on order closure is enforced by simply not offering a manual add-stock action anywhere in the mobile app. This keeps every inventory increase traceable to a closed order with an attached delivery document, at the cost of some convenience for operators who might otherwise expect a quick manual adjustment.

TAP-TO-ADD IN THE SCANNER INSTEAD OF AUTO-NAVIGATION

When a scan resolves to a creatable product, the scanner presents a result card with an explicit “Dodaj produkt” action rather than automatically navigating to a create form. Auto-navigation, combined with the camera’s rapid multi-fire callback, produced a fan-out of router pushes (the same problem the ref-guard addresses at a lower level). Keeping the action explicit and in-screen avoids that entire class of navigation bugs and keeps the operator in the scanning loop.

NOTIFICATION TEXT STAYS IN ENGLISH SERVER-SIDE

The notification service composes message text in English when it builds notifications from Kafka events. Localizing server-side would require the service to know each recipient’s locale and to carry parallel message templates, duplicating the client’s i18n responsibility on the server. The chosen tradeoff keeps the server simple and authoritative about facts (which order, which warehouse, what changed) while leaving presentation concerns to the clients.

Appendix A

QR payload schema

A product QR code encodes a single JSON object. The recognized fields are listed below; `t` or a `name` + `sku` pair marks the payload as a create-product payload, and any payload not matching this shape is treated as a bare SKU to look up.

FIELD	TYPE	REQUIRED	MEANING
<code>t</code>	string	no	Discriminator; <code>"product"</code> marks a create-product payload.
<code>name</code>	string	yes (for product)	Product name; also the default description when <code>description</code> is omitted.
<code>sku</code>	string	yes (for product)	Stock-keeping unit; checked for uniqueness via <code>getBySku</code> .
<code>description</code>	string	no	Free-text description; defaults to <code>name</code> .
<code>weight</code>	number	no	Unit weight.
<code>category</code>	string	no	Category name; resolved to an id, created on demand if missing.

Example payload:

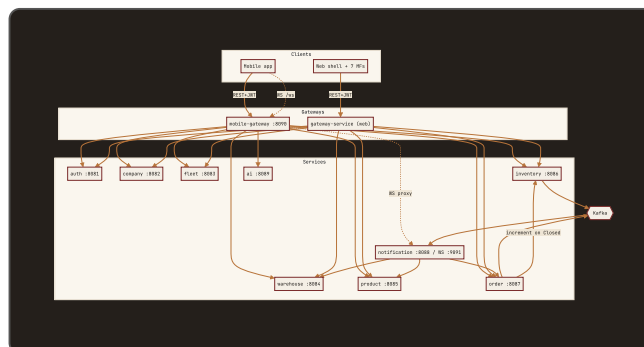
```
{  
  "t": "product",  
  "name": "Kladivo",  
  "sku": "KLA-001",  
  "description": "Jekleno kladivo 500g",  
  "weight": 0.7,  
  "category": "Orodje"  
}
```

Appendix B

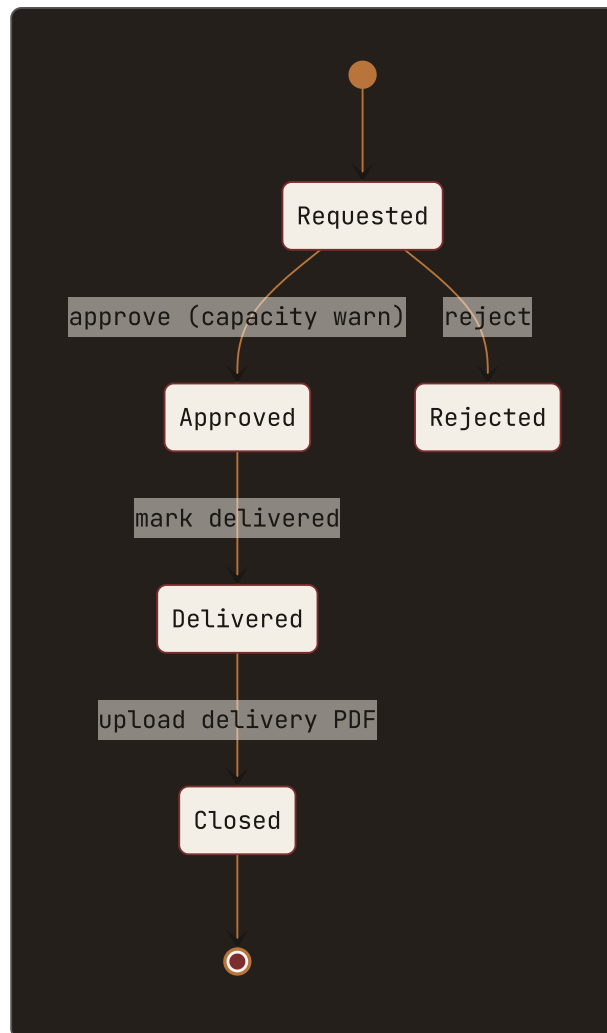
Mermaid diagrams (consolidated)

The diagrams below collect the system's structural and behavioral models in one place for quick reference.

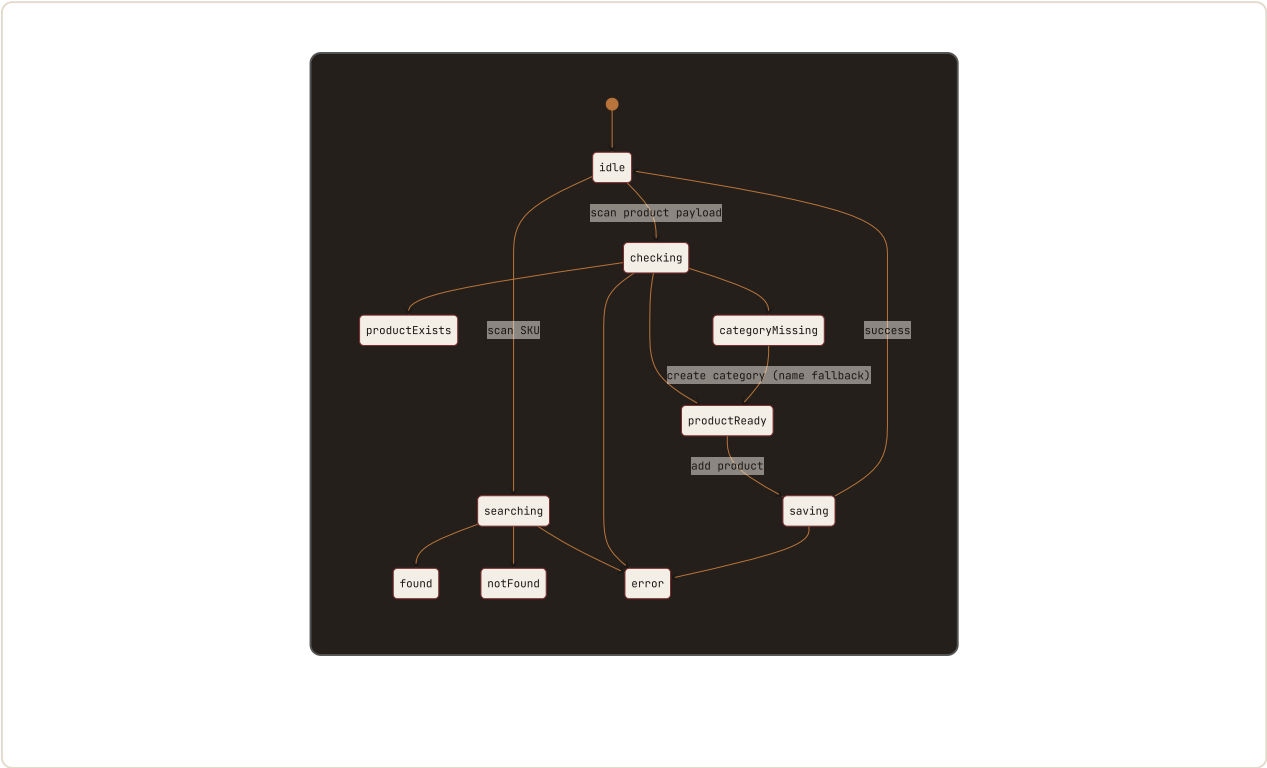
B.1 TOP-LEVEL ARCHITECTURE



B.2 ORDER STATE MACHINE



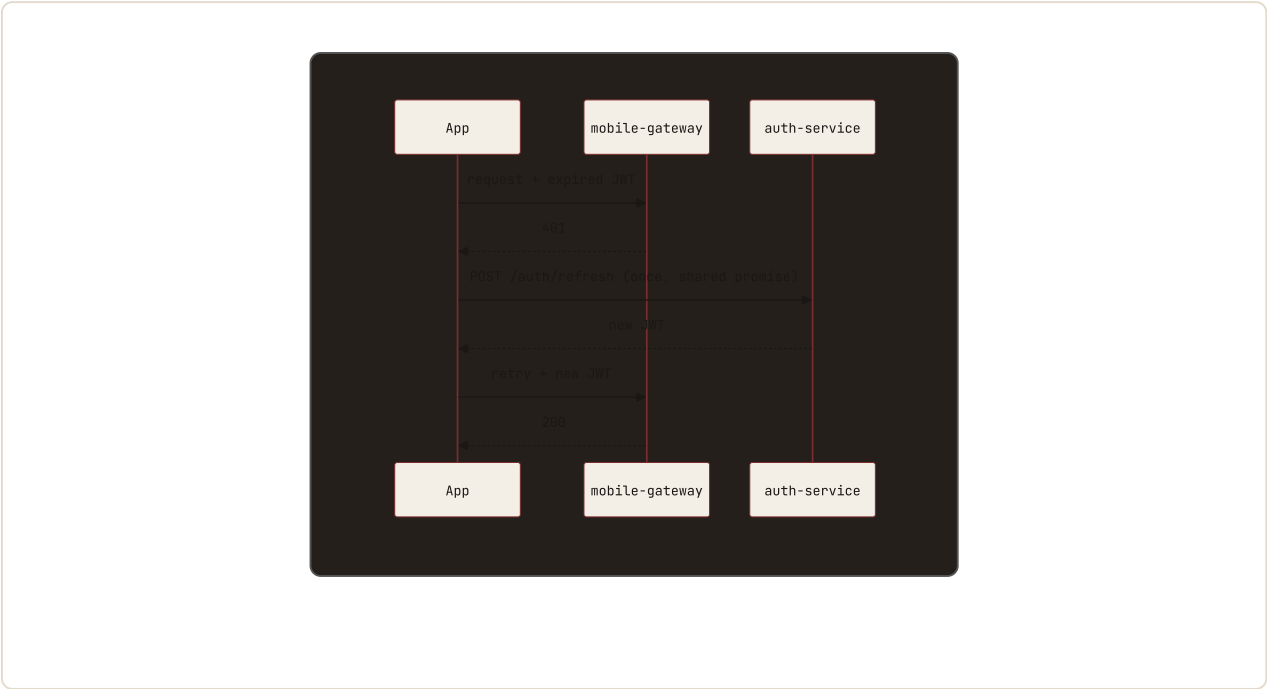
B.3 SCANNER STATE MACHINE



B.4 NOTIFICATION EVENT FLOW



B.5 JWT REFRESH SEQUENCE



Appendix C

Glossary

TERM	MEANING
BFF	Backend-for-frontend; the mobile gateway is a BFF tailored to the mobile client.
Closed (order)	Terminal order state; the only event that increases inventory stock.
Consume	Issuing/drawing down stock; a multipart operation requiring an attached PDF.
<code>concatMap</code>	RxJS operator that processes items in order without cancelling in-flight work; used for ordered persistence in the notification pipeline.
<code>getBySku</code>	Product-service lookup used by the scanner to test whether a SKU is already taken.
HMAC-SHA-256	The signing algorithm for the JWT; the gateway and notification service share the secret.
i18n	Internationalization; here, the Slovenian/English message system built on a live proxy.
JWT	JSON Web Token carrying identity and the authoritative <code>role</code> claim.
Module Federation	Webpack mechanism composing the web shell from seven independently built micro-frontends.
<i>Naročilo</i>	Slovenian for <i>order</i> .
<i>Obvestilo</i>	Slovenian for <i>notification</i> .
<code>requireManager</code>	Gateway middleware restricting the AI route to <code>MANAGER</code> and above.
<code>sl</code> proxy	The recursive Proxy that resolves every string access against the active locale at call time.
<i>Skener</i>	Slovenian for <i>scanner</i> .
<i>Skladišče</i>	Slovenian for <i>warehouse</i> .
SKU	Stock-keeping unit; a product's unique code.

TERM	MEANING
<code>useBarcode</code> ref-guard	A synchronous <code>useRef</code> lock preventing the camera's multi-fire callback from handling one scan many times.
<code>X-User-*</code> headers	<code>X-User-Id</code> / <code>Email</code> / <code>Role</code> stamped by the gateway after verifying the JWT, trusted by downstream services.
<i>Zaloga</i>	Slovenian for <i>stock / inventory</i> .